GOHACKS TALENT 2026 · 07 E 08/11 · GOGROUP HUB, FORTALEZA

Guia de preparação

Prompts, Claude Code, engenharia mínima, agentes, deploy, pitch e um plano de treino até o dia 06/11.

Individual	**~6h30**	**16h30**	**R$ 5 mil**
projeto e prêmio são seus	de construção no sábado	prazo da submissão	1º lugar (2º R$ 2 mil, 3º R$ 1 mil)

O que já se sabe

- O Gogroup é a holding da **Gocase** e da **Gobeaut**: e-commerce de acessórios e beleza. Os desafios vêm "das nossas marcas" e usam dados fictícios. Os exemplos do próprio app falam de reembolso, pedido com problema e reclamação de atendimento. Espere temas de operação de e-commerce: atendimento, logística, devolução, catálogo, marketing.
- O notebook vem pronto com **Claude** e a "infraestrutura de IA do grupo". A IA é liberada: **eles avaliam como você dirige, confere e corrige a IA**.
- O catálogo de desafios sai entre 03 e 06/11. A escolha é às 9h30 e as vagas por desafio são limitadas.
- **Confirme a presença no app até 02/11.**

Como a banca dá a nota

Critério	Peso	O que fazer para pontuar
Modelagem	40%	Fluxo de ponta a ponta funcionando, com **agente/automação** e pouca mão humana. Mostre que conferiu a saída da IA e corrigiu os erros. É o critério de desempate.
Adaptabilidade	20%	Aprenda rápido o tema da marca, use bem os dados fictícios e explique como seria com dados reais. Mude de rota quando algo não funcionar.
Conceito	20%	Uma persona com nome, uma dor real do negócio e um problema que dê para entender em 1 minuto.
Aderência cultural	20%	Chame mentor cedo, aceite feedback do pré-júri e ajuste o projeto na frente deles.

A estratégia em uma frase

Escolha **uma** dor operacional pequena e construa um agente que resolve o caso comum sozinho e manda o caso de risco para um humano. Meça quantos casos ele acertou nos dados fictícios e mostre isso funcionando ao vivo.

1 O sábado, hora a hora

O cronograma oficial está à esquerda. À direita, a meta que você precisa ter batido em cada horário. Se atrasar, corte escopo, nunca a demo.

Hora	Oficial	Sua meta
08h30	Credenciamento	Login no notebook, abrir o Claude, criar o repositório no GitHub, testar o Git e o deploy (veja a seção 6).
09h30	Escolha do desafio	Escolha em 5 min. Prefira o desafio cujos dados você entende e cuja demo dá para ver numa tela.
09h40–10h10	Mão na massa	Escreva o **SPEC.md**: 1 persona, 1 dor, 1 hipótese, o fluxo atual em 3 passos, o fluxo novo e a métrica. Valide com um mentor de negócio que passar na mesa.
10h10–12h	Mão na massa	Fluxo principal funcionando de ponta a ponta, ainda feio, com os dados fictícios. **Primeiro deploy antes do almoço.**
13h10–14h30	Bloco 2	Agente completo: ferramentas, decisão, saída estruturada e casos de risco indo para revisão humana. Rode em todos os casos fictícios e calcule a métrica.
14h30–15h20	Bloco 2	Interface apresentável, tela de antes e depois e números na tela. Grave o vídeo de 2 min (plano B).
15h20		**Feature freeze.** Depois disso, só correção de bug.
15h20–16h10		Slides em PDF (máx. 10), README e textos da submissão. Teste o link do app numa janela anônima.
16h10	Prazo às 16h30	**Submeta.** Não deixe para o aviso dos 30 minutos.
16h45–18h20	Pré-júri de 10 min	Pitch de 5 min mais perguntas. Enquanto espera, escreva o relatório individual.

Os textos da submissão (já se sabe o que pedem)

Campo	Como preencher
Problema (600 car.)	O que acontece hoje, quem sofre e quanto custa. Use um número dos dados fictícios.
Solução (800)	Passo a passo do ponto de vista de quem usa, não do código.
Como usou a IA (800)	Ferramentas, em que etapa, qual agente ou automação. Diga também **o que você conferiu e corrigiu**, por exemplo: "conferi 30 casos à mão, a IA errou 4 em X, ajustei o prompt".
Impacto (500)	Uma métrica de processo (tempo, taxa de acerto) e um impacto no negócio (custo, satisfação).
PDF, GitHub e app	O GitHub precisa ser **público**. Teste o app sem login numa janela anônima. Mande o vídeo de 2 min também.

Mantenha um `LOG.md` desde as 9h40

Uma linha por episódio: "11h05: a IA classificou devolução como troca em 3 de 20 casos; acrescentei exemplos no prompt; caiu para 0 de 20". Isso alimenta a nota de Modelagem, o campo "Como usou a IA", o relatório individual (que pergunta exatamente onde a IA errou) e a sabatina.

2 Prompt engineering

No sábado você escreve dois tipos de prompt: os que **dirigem o Claude enquanto ele programa** e os que ficam **dentro do seu produto**, que o agente usa para classificar, decidir e responder. Os princípios são os mesmos.

Os princípios (guia oficial da Anthropic)

1. **Seja claro e direto.** Imagine um colega competente que não sabe nada do seu projeto. O que ele precisaria saber? Escreva isso.
2. **Diga o porquê.** "Responda curto, porque o texto vai num WhatsApp" funciona melhor que "responda curto". Com o motivo, o modelo generaliza para casos que você não previu.
3. **Dê exemplos (few-shot).** Dois a cinco exemplos de entrada e saída, variados e com pelo menos um caso difícil. É o jeito mais barato de corrigir um classificador.
4. **Separe as partes com tags XML:** `<instrucoes>`, `<politica>`, `<pedido>`, `<exemplos>`. Assim o modelo não confunde dado com instrução.
5. **Peça uma saída estruturada.** JSON com campos fixos, validado no código. Sem isso, o agente não se encaixa no resto do sistema.
6. **Dê uma saída para a dúvida.** "Se faltar informação, devolva `"acao": "revisao_humana"` e o motivo." Isso evita alucinação e mostra o human-in-the-loop para a banca.
7. **Diga o que fazer em vez de só proibir.** Melhor "escreva em parágrafos curtos, sem listas" do que "não use markdown".
8. **Quebre tarefas grandes em etapas encadeadas.** Classificar, depois extrair, depois decidir, depois redigir. Cada etapa fica testável e você vê onde errou.
9. **Itere com dados.** Rode o prompt em todos os casos fictícios, olhe os erros, ajuste e rode de novo. Anote cada rodada no LOG.md.

Modelo de prompt para o agente do produto

```
Você faz a triagem de reclamações de clientes da [marca], um e-commerce de capinhas.
Seu resultado é lido por um sistema, não por uma pessoa: responda só com JSON válido.

<politica>
- Troca grátis até 7 dias após a entrega se o produto veio com defeito.
- Reembolso acima de R$ 300 sempre passa por um humano.
</politica>

<pedido>{dados do pedido vindos da ferramenta buscar_pedido}</pedido>
<mensagem>{texto do cliente}</mensagem>

Decida a ação seguindo a política. Se a mensagem for ambígua ou faltar dado,
use "revisao_humana" e explique o motivo em uma frase.

<exemplos> ... 3 casos, um deles ambíguo ... </exemplos>

Formato:
{"categoria": "defeito|atraso|arrependimento|outro",
 "acao": "trocar|reembolsar|responder|revisao_humana",
 "confianca": 0-1, "motivo": "...", "resposta_ao_cliente": "..."}
```

Prompts para dirigir o Claude enquanto programa

Em vez de	Escreva
"faz um app de devoluções"	"Leia SPEC.md e dados/pedidos.csv. Monte o fluxo: ler a mensagem, chamar `buscar_pedido`, decidir pela política e gravar o resultado. Sem biblioteca nova além de X. Rode `npm test` no final e me mostre a saída."
"tá dando erro, conserta"	Cole o erro inteiro e diga onde ele aparece: "Ao clicar em Processar, sai este erro. Escreva um teste que reproduza o erro e depois corrija."
"melhora o visual"	Cole uma captura da tela e diga o objetivo: "Isto vai para um telão. Quero os números de antes e depois em destaque e a lista de casos com status por cor. Siga o padrão de components/Card."
"pensa bem nisso"	Para um raciocínio difícil, escreva `ultrathink`, que pede um raciocínio mais profundo naquele turno. Para escopo maior, diga `be ambitious` e liste o que entra: estados de erro, vazio, loading.

Não sabe o que quer ainda? Peça para ser entrevistado

`"Quero resolver [dor]. Me entreviste com perguntas de múltipla escolha até conseguir escrever a especificação, depois grave em SPEC.md."` Em 10 minutos você tem a definição do pronto, que serve de base para o agente, os slides e a banca.

3 Vibecoding com controle: o fluxo com Claude Code

Vibecoding é deixar a IA escrever quase todo o código enquanto você descreve o que quer. Num hackathon de um dia é o caminho certo. O risco é perder o controle do que foi feito, e a banca vai perguntar como funciona. A regra: **a IA digita, você decide e confere.**

O ciclo: explorar → planejar → implementar → conferir → commitar

1. **Explorar:** peça para o Claude ler os dados e o enunciado e resumir o que entendeu. Corrija o entendimento antes de qualquer código.
2. **Planejar:** use o **plan mode** (`Shift+Tab`). Ele lê e propõe sem editar nada. Edite o plano (`Ctrl+G`) até concordar.
3. **Implementar** em fatias pequenas e verticais. Primeiro um caso funcionando de ponta a ponta, depois os outros.
4. **Conferir:** dê ao Claude algo que **passa ou falha**, como teste, build, script que compara a saída com o esperado ou captura de tela. "Rode os testes e continue até passar." Peça evidência: o comando e a saída, não só "deu certo".
5. **Commitar** a cada avanço. O commit é o seu ponto de restauração e o histórico que a banca vê no GitHub.

Comandos para saber de cor

Comando	Quando usar
`Esc`	O Claude saiu do rumo. Pare na hora e redirecione. Vale olhar o primeiro minuto de toda tarefa.
`Esc Esc` / `/rewind`	Voltar código e conversa a um ponto anterior. Isso deixa barato tentar a ideia mais ousada primeiro.
`/clear`	Corrigiu a mesma coisa duas vezes e continua errado? Limpe a conversa e escreva um prompt melhor com o que aprendeu. Uma terceira correção raramente resolve.
`@arquivo`	Colocar um arquivo inteiro no prompt, como os dados, o SPEC ou um componente de referência.
`/code-review`	Revisão do diff por um subagente que não viu a sua conversa. Faça uma por volta das 15h.
`/btw`	Pergunta rápida que não entra no histórico.
`CLAUDE.md`	Curto: o comando de teste, o comando de deploy e as regras que não dá para ver no código ("dados só de dados/, nunca chamar API real"). Não descreva a estrutura de pastas, porque o Claude lê o repositório sozinho.

O que você confere com os próprios olhos

Pode deixar a IA fazer

- Telas, CSS, componentes
- Leitura de CSV/JSON e boilerplate
- Testes a partir dos casos que você escolheu
- README, textos de interface
- Primeiro rascunho dos prompts do agente

Você confere sempre

- A lógica de decisão e as regras de negócio
- Os números que vão para o slide: recalcule pelo menos um
- Se a chave de API ficou fora do código e do front
- Se o fluxo mostrado na demo é real ou simulado (diga qual)
- Cada saída do agente nos 10–20 casos de teste

Antipadrões que custam o dia

- Pedir "o app inteiro" num prompt só. Sai muito código que você não entende e um bug no meio que ninguém acha.
- Colar erro sem contexto em loop. Depois de duas tentativas, pare, leia o erro, use `/clear` e reformule.
- Escrever em CAIXA ALTA e IMPORTANTE nos prompts. Os modelos atuais seguem uma linha simples, e a ênfase faz a regra ser aplicada demais.
- Trocar de stack às 14h. Use o que você já domina.
- Acrescentar feature depois das 15h20 em vez de polir a demo.

4 Agentes: o que a banca quer ver nos 40%

Um agente é um modelo de linguagem que usa **ferramentas** (funções que você escreve, como buscar pedido, consultar estoque ou enviar mensagem) em um **loop** até terminar a tarefa. A banca pergunta "faz o trabalho sozinha ou depende de mão humana?". Mostre autonomia no caso comum e controle no caso de risco.

Padrões, do mais simples ao mais autônomo

Padrão	Como funciona	Bom para
Cadeia de prompts	Etapas fixas: classificar → extrair → redigir. O código controla a ordem.	A base de quase tudo. Previsível e fácil de testar.
Roteador	O modelo classifica a entrada e manda para o tratamento certo.	Triagem de tickets e e-mails.
Agente com ferramentas	O modelo decide qual ferramenta chamar e quando parar.	Investigar um pedido, cruzar dados.
Avaliador-otimizador	Um segundo prompt revisa a saída do primeiro e pede correção.	Respostas ao cliente, descrições de produto.
Humano no loop	Baixa confiança ou alto valor vai para a fila de aprovação.	Reembolso, cancelamento. Mostra responsabilidade.

Para um projeto de um dia, o melhor costuma ser **cadeia ou roteador com uma ou duas ferramentas e um humano no loop**. Já é um agente de verdade, e cada etapa pode ser medida. Um agente totalmente livre é difícil de depurar em 6 horas.

Exemplo de arquitetura (tema provável de e-commerce)

```
mensagem do cliente
   |
   ▼
[1] classificar  ──► categoria + confiança         (prompt com exemplos, saída JSON)
   |
   ▼
[2] buscar_pedido(id) / buscar_rastreio(id)         (ferramentas: leem os CSVs fictícios)
   |
   ▼
[3] decidir pela política ──► trocar | reembolsar | responder | revisao_humana
   |                          (regra dura no código para o que não pode errar:
   |                           valor > R$ 300 → humano)
   ▼
[4] redigir resposta ──► [5] revisor confere o tom e a política
   |
   ▼
painel: resolvidos sozinhos x enviados ao humano, tempo médio, acerto
```

Medir é o que separa projeto de demo bonita

- Monte um **gabarito**: separe de 20 a 30 casos fictícios e anote à mão a resposta certa. Leva 15 minutos.
- Rode o agente em todos e calcule o acerto. Ajuste o prompt e rode de novo. "Acerto foi de 70% para 93% depois de acrescentar 3 exemplos" é a frase ideal para o slide e para a sabatina.
- Escolha **um número de negócio**: "Hoje cada ticket leva ~8 min. Com o agente, 80% saem em segundos e o time só revisa os 20% de risco." Diga que é estimativa e de onde veio.
- Você já faz isso nos seus projetos, como o WAPE contra o modelo da empresa e o gabarito da correção de listas. Use a mesma lógica: linha de base, sua solução e a diferença.

Regra dura no código, julgamento no prompt

O que não pode errar (limite de valor, prazo legal, dado obrigatório) vai num `if` no código, não no prompt. O prompt fica com o que exige leitura de texto. Explicar essa divisão na sabatina mostra maturidade de engenharia.

5 Engenharia de software para um dia

Ninguém espera arquitetura de produção. Espera-se que funcione na demo, que o código seja legível no GitHub e que você saiba explicar. O mínimo que entrega isso:

Stack: use o que você já domina

Opção	Quando escolher
React + TypeScript (Vite) + função serverless	Você já fez o Songe com React 19 e TS. É o caminho para um app bonito publicado na Vercel. A chamada ao modelo fica numa rota `/api`, nunca no navegador.
Python + Streamlit	Se o desafio for mais de dados ou análise. Painel em minutos, deploy no Streamlit Community Cloud. Menos bonito, muito mais rápido.
Next.js	Só se já estiver confortável. Front e API juntos na Vercel.

Pergunte no credenciamento o que é a "infraestrutura de IA do grupo": um endpoint próprio, uma chave da API da Anthropic, ou só o Claude no navegador ou no Claude Code. Isso define como o seu app chama o modelo.

Estrutura que cabe no dia

```
projeto/
  README.md        o problema, como rodar, o link do app, a métrica
  SPEC.md          persona, dor, fluxo, definição do pronto
  LOG.md           cada erro da IA e o que você corrigiu
  CLAUDE.md        comandos de teste e deploy, regras do projeto
  dados/           CSV/JSON fictícios do desafio (só estes!)
  src/agente/      prompts.ts, ferramentas.ts, pipeline.ts
  src/ui/          telas
  tests/           gabarito.test.ts: roda os 20 casos e checa o acerto
```

Práticas que valem o tempo

- **Isole o núcleo** (o pipeline do agente) numa função pura: `processar(mensagem, pedido) → decisão`. Testável sem interface e reaproveitável no script de avaliação.
- **Valide a saída do modelo.** Faça o parse do JSON com tratamento de erro (zod no TS, pydantic no Python). Se vier inválido, tente uma vez de novo e depois mande para revisão humana. O app não pode quebrar na frente da banca.
- **Guarde em cache as respostas da demo.** Salve as saídas dos casos que você vai mostrar. Se a rede ou a API cair no pitch, a demo continua. Diga que é cache.
- **Segredos em variável de ambiente** (`.env` no `.gitignore`). Repositório público com chave exposta é o erro mais comum de hackathon.
- **Commits pequenos e frequentes**, com mensagens que contam a história ("agente classifica 20 casos com 93%").
- **Testes só no que importa:** o gabarito do agente e as regras duras. Não gaste tempo testando a interface.
- **Nada de dados reais.** O regulamento proíbe e você assina termo de confidencialidade. Na declaração da submissão você afirma que fez tudo no evento, então leve conhecimento e templates na cabeça, não código pronto do projeto.

6 Produção e deploy

O link do app é opcional, mas a banca valoriza "funciona de verdade". Deixe o deploy pronto no começo do dia, não às 16h.

Checklist de deploy (treine em casa até sair em 10 min)

1. Repositório público no GitHub com o primeiro commit às 8h50.
2. Conecte o repositório na **Vercel** (ou no Streamlit Cloud). Cada `git push` publica sozinho.

3. Cadastre a chave de API em **Environment Variables** no painel, nunca no código.
4. Faça a chamada ao modelo numa função de servidor (`api/agente.ts`). O navegador só chama a sua rota.
5. Abra o link numa **janela anônima**. Precisa abrir sem login nenhum.
6. Abra no celular também. A banca pode abrir o QR Code.

O que costuma quebrar na hora

Sintoma	Causa e correção
Funciona local, quebra no ar	Variável de ambiente não cadastrada na Vercel, ou caminho de arquivo relativo. Leia os logs da função no painel.
Erro de CORS	O front chamando a API do modelo direto. Passe pela sua rota `/api` .
Função passa do tempo limite	Várias chamadas em sequência. Rode os casos em lote no build ou use cache, e mostre um ao vivo.
Tela branca	Erro de build ignorado. Rode `npm run build` localmente antes do push.

Plano B obrigatório

- Vídeo de até 2 min com a solução funcionando, gravado às 15h, no YouTube não listado ou no Drive aberto.
- Capturas da tela ou um GIF da demo dentro do PDF dos slides.
- Modo demo com respostas em cache, se a API falhar.

7 Pitch de 5 minutos e sabatina

Tempo	Bloco	Conteúdo
0:00–0:15	Abertura	Você, o projeto e o desafio.
0:15–1:20	Dor e fluxo atual	Persona com nome: "A Ana, do atendimento da Gocase, abre 3 sistemas para cada reclamação..." e onde está o atrito.
1:20–2:10	Solução	"Minha solução é [nome]: um agente de IA que [ação], para que [resultado]. Deixa eu mostrar."
2:10–3:30	**Demo**	Mostre em vez de descrever. Um caso comum resolvido sozinho e um de risco indo para o humano.
3:30–4:00	Métrica	"Antes: 8 min. Depois: segundos. Acerto de 93% em 30 casos conferidos."
4:00–5:00	Implantação	O que muda no dia a dia, a primeira dependência (ex.: acesso à API de pedidos) e o que você faria na segunda-feira.

Perguntas prováveis: como funciona tecnicamente (2 frases e mostre na tela) · quem usaria · o que falta para ir para produção (seja honesto) · como medir sucesso · **onde a IA errou e como você percebeu** (o LOG.md responde). Se não souber: "Não cheguei lá ainda, mas o próximo passo seria...".

Slides: no máximo 10, até 3 elementos por slide, exportados em PDF. Se precisar cortar, corte texto, não a demo.

8 Skills do Claude para o hackathon

Skills são pacotes de instruções (um `SKILL.md`) que o Claude carrega quando a tarefa pede. **Atenção:** o notebook do evento vem configurado pela organização e você pode não conseguir instalar nada. Treine com elas em casa para aprender os métodos, porque o método vai com você mesmo que a skill não esteja lá.

As mais recomendadas (listas de 2026 e repositório oficial da Anthropic)

Skill	Origem	Para que serve no sábado
frontend-design	anthropics/skills	Interfaces com visual próprio, sem cara de template. A primeira escolha para o app ficar apresentável no telão. **já instalada**
superpowers	obra/superpowers	Pacote de método: brainstorming, writing-plans, executing-plans, TDD, systematic-debugging, code review. É o ciclo da seção 3 em forma de skill.
webapp-testing	anthropics/skills	O Claude abre o app com Playwright, clica e confere a tela. Pega o bug da demo antes da banca.
pptx / pdf	anthropics/skills	Gerar os slides do pitch e exportar em PDF. **já instaladas**
skill-creator	anthropics/skills	Criar uma skill sua. Treine em casa: uma skill "hackathon" com o seu SPEC, LOG e checklist.
mcp-builder	anthropics/skills	Construir servidores MCP. Só vale se o desafio pedir integração com sistemas. Para um dia, provavelmente é excesso.

Que você já tem instaladas e combinam com o evento

Skill	Uso
`mattpocock-skills:grilling`	O Claude te bombardeia de perguntas sobre a ideia. É o ensaio da sabatina e a validação do problema às 10h.
`mattpocock-skills:prototype`	Protótipo descartável para testar se a interface e o fluxo fazem sentido antes de construir.
`mattpocock-skills:tdd`	Escrever o teste do gabarito antes do agente. É o "algo que passa ou falha" da seção 3.
`kill-ai-slop` e `web-design-guidelines`	Tirar a cara de "feito por IA" da interface: gradiente roxo, emoji, card dentro de card. A banca vê muitos projetos com o mesmo visual.
`vercel-react-best-practices`	Se for React/Next na Vercel.
`dataviz`	O gráfico de antes e depois no painel e no slide.
Comandos `/code-review` e `/simplify`	Revisão independente do diff às 15h e limpeza antes de deixar o repositório público.

Instalar em casa

```
/plugin marketplace add anthropics/skills
/plugin install example-skills@anthropic-agent-skills     # frontend-design, webapp-testing, skill-creator...
/plugin install document-skills@anthropic-agent-skills    # pptx, pdf, docx, xlsx
/plugin install superpowers@claude-plugins-official
```

Comandos conforme as fontes de out/2026. Reinicie o Claude Code depois de instalar. As listas de "melhores skills" divergem e muitas são conteúdo de SEO. As da Anthropic e a superpowers foram as mais citadas.

9 Plano de treino: 29/10 a 06/11

Dia	O que fazer
Qua 29/10	**Confirmar presença no app** (prazo 02/11). Ler o toolkit, os critérios e o FAQ do app. Instalar as skills da seção 8.
Qui 30/10	Treino de deploy: projeto vazio React+TS com rota `/api` chamando o Claude, publicado na Vercel com variável de ambiente. Cronometre. A meta é 10 min.
Sex 31/10	Treino de prompt: invente 20 reclamações de e-commerce, monte o gabarito, escreva o classificador com saída JSON, meça o acerto e ajuste duas vezes. Anote tudo num LOG.md.
Sáb 01/11	**Simulado completo de 6h30**, no horário real (9h40–16h10), com um tema de e-commerce que você não escolheu antes, por exemplo "atrasos de entrega da Gobeaut". Siga as metas da seção 1 e submeta para você mesmo.
Dom 02/11	Revisar o simulado: onde perdeu tempo? Gravar o pitch de 5 min e assistir. Usar `grilling` para ensaiar a sabatina.
Seg–Ter 03–04/11	Ler o **catálogo de desafios** assim que sair. Para os 2–3 que mais combinam com você: persona, dor, fluxo atual, ideia de agente e métrica. Pesquise a operação da Gocase e da Gobeaut (site, avaliações de clientes, Reclame Aqui) para entender as dores reais.
Qua–Qui 05–06/11	Segundo simulado, mais curto (3h), com um dos desafios do catálogo usando dados que você inventar. Ler o e-mail de véspera. Separar documento, carregador, garrafinha e casaco.
Sex 06/11	Descanso. Dormir cedo, o credenciamento é às 8h30.

Cartão de bolso (para levar na cabeça)

1. Uma persona, uma dor, uma métrica. SPEC.md até 10h10.
2. Ponta a ponta feio antes do almoço, e deploy também.
3. Agente = etapas + ferramentas + humano no caso de risco.
4. Gabarito de 20–30 casos. Acerto medido, prompt ajustado, número no slide.
5. LOG.md de cada erro da IA. Vale nota, alimenta o relatório e responde a sabatina.
6. Regra dura no código, julgamento no prompt, JSON validado.
7. Duas correções falharam? `/clear` e um prompt melhor.
8. Freeze às 15h20, vídeo de plano B, submissão às 16h10.
9. Chame mentor cedo e aceite o feedback: são 20% da nota.

Fontes

- App do evento: gohacks-talent-evento.devgogroup.com (cronograma, critérios, toolkit, FAQ e campos da submissão, via API pública do app)
- Anthropic: Best practices for Claude Code, code.claude.com/docs/en/best-practices
- Anthropic: Prompt engineering overview e Prompting best practices, platform.claude.com/docs/en/prompt-engineering
- Anthropic: repositório de skills, github.com/anthropics/skills
- Tella: The 12 Best Claude Code Skills in 2026, tella.com/blog/best-claude-code-skills
- obra/superpowers, github.com/obra/superpowers
- Diário do Nordeste: grupo cearense dono da Gocase investe R$ 50 milhões (Gogroup, Gocase e Gobeaut)